



---

ASIC Verification  
EE5213  
Exercise 1 Report  
Counter Design and  
Parameterized Stack Verification

---

**Student:** Nguyen Thai Thanh Binh - 2570175

**Instructor:** TS. Tran Hoang Linh

April 1, 2026

# 1 Exercise 1: Counter Design and Verification

## 1.1 Objective

Design a 5-bit synchronous counter based on the specifications provided and write a testbench to verify its correctness against a target timing trace.

## 1.2 Design Specifications

- The counter is clocked on the rising edge of `clk`.
- `rst` is active high and asynchronously resets the output to 0.
- `cnt_in` and `cnt_out` are 5-bit signals.
- If `load` is high, the counter is loaded from `cnt_in` (takes priority over `en`).
- Otherwise, if `en` is high, `cnt_out` is incremented.

## 1.3 Implementation (counter.v)

```
1 'timescale 1ns / 1ps
2
3 module counter (
4     input clk,
5     input rst,
6     input load,
7     input en,
8     input [4:0] cnt_in,
9     output reg [4:0] cnt_out
10 );
11     always @(posedge clk or posedge rst) begin
12         if (rst) begin
13             cnt_out <= 5'b00000;
14         end else if (load) begin
15             cnt_out <= cnt_in;
16         end else if (en) begin
17             cnt_out <= cnt_out + 1;
18         end
19     end
20 endmodule
```

## 1.4 Testbench (counter\_tb.v)

To match the precise timestamps detailed in the exercise, the testbench drives inputs securely synchronized around the clock edges.

```
1 'timescale 1ns / 1ps
2
3 module counter_tb;
4     reg clk;
5     reg rst, load, en;
6     reg [4:0] cnt_in;
7     wire [4:0] cnt_out;
```

```

8
9     counter uut (.clk(clk), .rst(rst), .load(load), .en(en), .cnt_in(
cnt_in), .cnt_out(cnt_out));
10
11     initial begin
12         clk = 0;
13         forever #5 clk = ~clk;
14     end
15
16     initial begin
17         rst = 1; load = 0; en = 0; cnt_in = 0;
18
19         #12; rst = 0; load = 1; en = 1; cnt_in = 5'b10101;
20         #8; $display("At time %0t rst=%b load=%b en=%b cnt_in=%b
cnt_out=%b",
21             $time, rst, load, en, cnt_in, cnt_out);
22
23         #2; rst = 0; load = 1; en = 1; cnt_in = 5'b01010;
24         #8; $display("At time %0t rst=%b load=%b en=%b cnt_in=%b
cnt_out=%b",
25             $time, rst, load, en, cnt_in, cnt_out);
26
27         #2; rst = 0; load = 1; en = 1; cnt_in = 5'b11111;
28         #8; $display("At time %0t rst=%b load=%b en=%b cnt_in=%b
cnt_out=%b",
29             $time, rst, load, en, cnt_in, cnt_out);
30
31         #2; rst = 1; load = 1; en = 1; cnt_in = 5'b11111;
32         #8; $display("At time %0t rst=%b load=%b en=%b cnt_in=%b
cnt_out=%b",
33             $time, rst, load, en, cnt_in, cnt_out);
34
35         #2; rst = 0; load = 1; en = 1; cnt_in = 5'b11111;
36         #8; $display("At time %0t rst=%b load=%b en=%b cnt_in=%b
cnt_out=%b",
37             $time, rst, load, en, cnt_in, cnt_out);
38
39         #2; rst = 0; load = 0; en = 1; cnt_in = 5'b11111;
40         #8; $display("At time %0t rst=%b load=%b en=%b cnt_in=%b
cnt_out=%b",
41             $time, rst, load, en, cnt_in, cnt_out);
42
43         #10; $finish;
44     end
45 endmodule

```

## 1.5 Simulation Results

The simulation log matches the specified requirements at the detailed time points.

```

1 At time 20000 rst=0 load=1 en=1 cnt_in=10101 cnt_out=10101
2 At time 30000 rst=0 load=1 en=1 cnt_in=01010 cnt_out=01010
3 At time 40000 rst=0 load=1 en=1 cnt_in=11111 cnt_out=11111
4 At time 50000 rst=1 load=1 en=1 cnt_in=11111 cnt_out=00000
5 At time 60000 rst=0 load=1 en=1 cnt_in=11111 cnt_out=11111
6 At time 70000 rst=0 load=0 en=1 cnt_in=11111 cnt_out=00000

```

## 2 Exercise 2: Stack Design and Verification

### 2.1 Objective

Create a parameterized LIFO (Last-In-First-Out) Stack design, present testing scenarios, and write a testbench to verify edge cases.

### 2.2 Design Specifications

- Parameterized for WIDTH and DEPTH. DEPTH is dynamically managed via DEPTH\_LOG2 (power of 2 constraint).
- Asynchronous high-active reset (`rst`).
- `en_wr` (write) and `en_rd` (read) synchronous to the rising edge of `clk`.
- Data written later corresponds to earlier reads (LIFO constraint).
- Ignores `en_wr` attempts when `full` flag is set.
- Ignores `en_rd` attempts when `empty` flag is set.

### 2.3 Implementation (stack.v)

```
1  `timescale 1ns / 1ps
2
3  module stack #(
4      parameter WIDTH = 8,
5      parameter DEPTH_LOG2 = 3
6  ) (
7      input clk, rst, en_wr, en_rd,
8      input [WIDTH-1:0] data_wr,
9      output reg [WIDTH-1:0] data_rd,
10     output reg full, empty
11 );
12     localparam MAX_DEPTH = 1 << DEPTH_LOG2;
13     reg [WIDTH-1:0] mem [0:MAX_DEPTH-1];
14     reg [DEPTH_LOG2:0] sp;
15
16     always @(posedge clk or posedge rst) begin
17         if (rst) begin
18             sp <= 0; full <= 0; empty <= 1; data_rd <= 0;
19         end else begin
20             if (en_wr && !full) begin
21                 mem[sp] <= data_wr;
22                 sp <= sp + 1;
23                 empty <= 0;
24                 if (sp + 1 == MAX_DEPTH) full <= 1;
25             end else if (en_rd && !empty) begin
26                 data_rd <= mem[sp - 1];
27                 sp <= sp - 1;
28                 full <= 0;
29                 if (sp - 1 == 0) empty <= 1;
30             end
31         end
32     end
```

```

32     end
33 endmodule

```

## 2.4 Testbench (stack\_tb.v)

```

1  `timescale 1ns / 1ps
2
3  module stack_tb;
4      parameter WIDTH = 8;
5      parameter DEPTH_LOG2 = 2; // Testing depth 4
6
7      reg clk, rst, en_wr, en_rd;
8      reg [WIDTH-1:0] data_wr;
9      wire [WIDTH-1:0] data_rd;
10     wire full, empty;
11
12     stack #(.WIDTH(WIDTH), .DEPTH_LOG2(DEPTH_LOG2)) uut (
13         .clk(clk), .rst(rst), .en_wr(en_wr), .en_rd(en_rd),
14         .data_wr(data_wr), .data_rd(data_rd), .full(full), .empty(empty)
15     );
16
17     initial forever #5 clk = ~clk;
18
19     initial begin
20         rst = 1; en_wr = 0; en_rd = 0; data_wr = 0;
21         #15; rst = 0; #10;
22
23         $display("Pushing elements...");
24         push_data(8'h11); push_data(8'h22); push_data(8'h33); push_data
(8'h44);
25
26         $display("Trying to push when full...");
27         push_data(8'h55);
28         if (full) $display("Stack is full, correctly blocked push.");
29
30         $display("Popping elements...");
31         pop_data(); pop_data(); pop_data(); pop_data();
32
33         $display("Trying to pop when empty...");
34         pop_data();
35         if (empty) $display("Stack is empty, correctly blocked pop.");
36
37         #20 $finish;
38     end
39
40     task push_data(input [WIDTH-1:0] val);
41     begin
42         @(posedge clk); en_wr = 1; data_wr = val;
43         @(posedge clk); en_wr = 0;
44         $display("Pushed %h | full=%b empty=%b", val, full, empty);
45     end
46     endtask
47
48     task pop_data();
49     begin

```

```

50         @(posedge clk); en_rd = 1;
51         @(posedge clk); en_rd = 0;
52         $display("Popped %h | full=%b empty=%b", data_rd, full, empty);
53     end
54     endtask
55 endmodule

```

## 2.5 Simulation Results

```

1 Pushing elements...
2 Pushed 11 | full=0 empty=0
3 Pushed 22 | full=0 empty=0
4 Pushed 33 | full=0 empty=0
5 Pushed 44 | full=1 empty=0
6 Trying to push when full...
7 Pushed 55 | full=1 empty=0
8 Stack is full, correctly blocked push.
9 Popping elements...
10 Popped 44 | full=0 empty=0
11 Popped 33 | full=0 empty=0
12 Popped 22 | full=0 empty=0
13 Popped 11 | full=0 empty=1
14 Trying to pop when empty...
15 Popped 11 | full=0 empty=1
16 Stack is empty, correctly blocked pop.

```